Well formed forumlas

A formula G is called well formed (WFF) if

$$G := At \mid (G_1 \odot \ldots \odot G_n) \mid (\neg G_1) \mid \top \mid \bot \qquad \odot \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$$

where At is an atomic proposition (also called atom or bool. variable) and G_i are themselfs WFFs

Entailment

Given a finite set of formulas $\Gamma = \{G_1, G_2, \ldots\}$:

$$\forall I : (I(\Gamma) = 1 \iff \forall G \in \Gamma : I(G))$$

$$\Gamma \models Q \iff \forall I : (\forall G \in \Gamma : I(G) = 1) \implies I(Q) = 1$$

$\Gamma \models Q \iff \Gamma \wedge \neg$ is unsat

Note that $\neg(\Gamma \rightarrow Q) = (\Gamma \wedge \neg Q)$

Connectives Priority (to avoid parenthesis):

1.$\neg$, 2.$\wedge$, 3.$\vee$, 4. $\rightarrow$, 5. $\leftrightarrow$

– **Example:**

$$\neg A \wedge B \rightarrow C \vee D \leftrightarrow E$$

is read as: $(((\neg A) \wedge B) \rightarrow (C \vee D) \leftrightarrow E)$

Clauses and Literals

– A literal is an atom p or its negation $\neg p$. Literal $\overline{p} = \neg p$

– A clause is a disjunction of literals $\bigvee_{i=1}^{n} l_i$

– An empty clause $\square$ ($n = 0$) is always negative

– A clause with one literal ($n = 1$) is a unit clause

– A clause with at most one positive literal is a horn clause

Subformulas

Given a formula G, every WFF Q in G is called a subformula

For a WFF $G = Q_i \odot \ldots \odot Q_n$

– $G|_\varepsilon = G$

– $G|_i = Q_i$

– if $Q_i = Q'_1 \odot \ldots \odot Q'_m$ then $G|_{i,j} = Q'_j$

– let π be a position i,j, ... where a subformula occurse, then $A|_\pi = B$ denotes that subformula B occurs in A at position π

Equivalent Replacement + Rewrite Rules

Given a formula G with subformula Q_1 denoted as $G[Q_1]$: $G[Q_2]$ would then denote the formula G where subformula Q_1 has been replaced with Q_2

– **Lemma:** $Q_1 \leftrightarrow Q_2 \iff G[Q_1] \leftrightarrow G[Q_1 / Q_2]$

– **Lemma:** $I \models p \iff I \models p \leftrightarrow \top$

$I \not\models p \iff I \models p \leftrightarrow \bot$

– **Formula evaluation** (using rewrite rule system)

```
procedure evaluate(G, I)
input: formula G, interpretation I
output: the value I(G)
begin
  forall atoms p occurring in G
    if I ⊨ p
      then replace all occurrences of p in G by ⊤;
      else replace all occurrences of p in G by ⊥;
  rewrite G into a normal form using the rewrite rules,
  if G = ⊤ then return 1 else return 0
end
```

Monotonic Replacement

Given a formula G with subformula Q_1 at position π denoted as $G[Q_1]_\pi$: $G[Q_2]_\pi$ would then denote the formula G where subformula Q_1 at position π has been replaced with Q_2

– **Lemma:** Given formulas G, Q_1, Q_2 where $I \models Q_1 \rightarrow Q_2$ and $G[Q_1]$

if $pol(A, \pi) = 1$ then $I \models G[Q_1]_\pi \rightarrow G[Q_2]_\pi$

if $pol(A, \pi) = -1$ then $I \models G[Q_2]_\pi \rightarrow G[Q_1]_\pi$

– **Theorem:** Given G, Q_1, Q_2 such that $Q_1 \rightarrow Q_2$ is valid and $Q_1 = G|_\pi$

if $pol(G, \pi) = 1$ then $G[Q_1]_\pi \rightarrow G[Q_2]_\pi$

if $pol(G, \pi) = 1$ then $G[Q_2]_\pi \rightarrow G[Q_1]_\pi$

CNF Transformations

$$A \leftrightarrow B \implies (\neg A \vee B) \wedge (\neg B \vee A)$$
$$A \rightarrow B \implies \neg A \vee B$$
$$\neg(A \wedge B) \implies \neg A \vee \neg B$$
$$\neg(A \vee B) \implies \neg A \wedge \neg B$$
$$\neg\neg A \implies A$$
$$\left(\bigwedge_{i=1}^{m} A_i\right) \vee \bigvee_{j=1}^{n} B_j \implies \bigwedge_{i=1}^{n} \left(A_i \vee \bigvee_{j=1}^{n} B_j\right)$$

Splitting

Given a formula G :

$G_p^{\top/\bot}$ denotes the formula where all occurences of p in formula G are replaced by $\top/\bot$

– **Lemma:** $I \models p \iff I(G) = I(G_p^\top)$

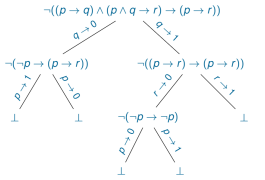
$I \not\models p \iff I(G) = I(G_p^\top)$

– **Theorem:** G is sat $\iff G_p^\top$ is sat $\vee\, G_p^\bot$ is sat

Splitting Algorithm

```
procedure split(G)
parameters: function select
input: formula G
output: "satisfiable" or "unsatisfiable"
begin
  G := simplify(G)
  if G = ⊤ then return "satisfiable"
  if G = ⊥ then return "unsatisfiable"
  (p, b) := select(G)
  case b of
  1 ⇒
    if split(G_p^⊤) ="satisfiable"
      then return "satisfiable"
      else return split(G_p^⊥)
  0 ⇒
    if split(G_p^⊥) ="satisfiable"
      then return "satisfiable"
      else return split(G_p^⊤)
end
```

– **Example:**



Polarity

The polariy of a formula G and its subformulas is defined as :

– $pol(G, \varepsilon) = 1$ (this means that the input formula always needs to have polarity of 1)

– if $G|_\pi = \bigwedge_{i=1}^{n} Q_i$ then

$$pol(G, \pi.i) = pol(G, \pi)$$

– if $G|_\pi = \bigvee_{i=1}^{n} Q_i$ then

$$pol(G, \pi.i) = -pol(G, \pi)$$

– if $G|_\pi = Q_1 \rightarrow Q_2$ then

$$pol(G, \pi.1) = -pol(G, \pi)$$
$$pol(G, \pi.2) = pol(G, \pi)$$

– if $G|_\pi = Q_1 \leftrightarrow Q_2$ then

$$pol(G, \pi.1) = pol(G, \pi.2) = 0$$

– if $G|_\pi = \neg Q$ then

$$pol(G, \pi.1) = -pol(G, \pi)$$

– if $G[Q]_\pi$ and $pol(G, \pi) = 1$ then Q is a positive occurence

– if $G[Q]_\pi$ and $pol(G, \pi) = -1$ then Q is a negative occurence

– **Coloring Algorithm**

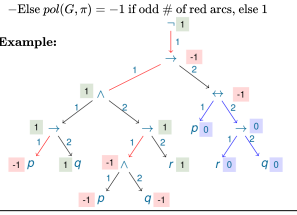
–Color in blue all arcs below in $\leftrightarrow$

–Color in red all uncolored arcs going down from $\neg$ or left side of $\rightarrow$

–If $G|_\pi$ has at least one blue arc, then $pol(G, \pi) = 0$

–Else $pol(G, \pi) = -1$ if odd # of red arcs, else 1

– **Example:**



Pure Atoms

Given a formula G, an atom p is pure if all occurences of p in A have the same polarity $\neq 0$

– **Lemma:** Given $I \models G$ and G has pure atom p. If p

–has only positive occurences then $I + \{p \mapsto 1\} \models G$

G is satisfiable $\iff G = G_p^\top$

–has only negative occurences then $I + \{p \mapsto 0\} \models G$

G is satisfiable $\iff G = G_p^\bot$

Pure Literals

Given a literal L occuring in a set of clauses S. L is called pure if S contains no clause $C_i = \left(\overline{L} \vee \bigvee_j L_{i,i}\right)$.

– **Optimization:**

If a pure literal L occurs in S then all clauses containing L can be satisfied by assigining the suitable truth value to L and can thus be removed.

Unit Propagation

Given a set of clauses $S = L \wedge \bigwedge_i C_i$.

Let S' be the set of clauses where all clauses C containing L are removed and all $C_i = \left(\overline{L} \vee \bigvee_j L_{i,j}\right)$ are replaced by $C'_i = \left(\bigvee_j L_{i,j}\right)$. Then it holds that $S \iff S' \cup \{L\}$

Definition Translation + Clausal Form

– **Naming Lemma**

Given a set of formulas S and a formula B. Let n be variable not occuring in S nor in $B \in S$. Let S' be the set of formulas where B has been replaced by n. Then S is sat $\iff S' \cup \{n \leftrightarrow B\}$ is sat.

– A clausal form of a formula A is a set of clauses A' where:

$A \iff A'$

– A clausal form of a set of formulas S is a set of clauses S' where:

$S \iff S'$

– **Example of Defintional Clause Form Translation**

(unoptimized version)

	subformula	definition	clauses
n_1	$\neg((p \rightarrow q) \wedge (p \wedge q \rightarrow r) \rightarrow (p \rightarrow r))$	$n_1 \leftrightarrow \neg n_2$	$\neg n_1 \vee \neg n_2$ $n_1 \vee \ \ n_2$
n_2	$(p \rightarrow q) \wedge (p \wedge q \rightarrow r) \rightarrow (p \rightarrow r)$	$n_2 \leftrightarrow (n_3 \rightarrow n_7)$	$\neg n_2 \vee \neg n_3 \vee n_7$ $n_3 \vee \ \ n_2$ $\neg n_7 \vee \ \ n_2$
n_3	$(p \rightarrow q) \wedge (p \wedge q \rightarrow r)$	$n_3 \leftrightarrow (n_4 \wedge n_5)$	$\neg n_3 \vee \ \ n_4$ $\neg n_3 \vee \ \ n_5$ $\neg n_4 \vee \neg n_5 \vee n_3$
n_4	$p \rightarrow q$	$n_4 \leftrightarrow (p \rightarrow q)$	$\neg n_4 \vee \neg p \vee q$ $p \ \vee \ \ n_4$ $\neg q \ \vee \ \ n_4$
n_5	$p \wedge q \rightarrow r$	$n_5 \leftrightarrow (n_6 \rightarrow r)$	$\neg n_5 \vee \neg n_6 \vee r$ $n_6 \vee \ \ n_5$ $\neg r \ \vee \ \ n_5$
n_6	$p \wedge q$	$n_6 \leftrightarrow (p \wedge q)$	$\neg n_6 \vee \ \ p$ $\neg n_6 \vee \ \ q$ $\neg p \ \vee \neg q \ \vee n_6$
n_7	$p \rightarrow r$	$n_7 \leftrightarrow (p \rightarrow r)$	$\neg n_7 \vee \neg p \ \vee r$ $p \ \vee \ \ n_7$ $\neg r \ \vee \ \ n_7$

Inferences

Inferences δ are rules of form:

$\delta = \dfrac{\varphi \qquad \psi_1, \ldots, \psi_n}{\chi}$, where:

– φ is the prerequite of δ

– $\psi_1, \ldots, \psi_n$ are justifications of δ

– χ is the consquence of of δ

One can also write: $\underbrace{\{\varphi, \psi_1, \ldots, \psi_n\}}_{KB} \vdash \underbrace{\chi}_{A}$

– **Modus Ponens:**

$\dfrac{A \rightarrow B \qquad A}{B} MP$

Read as: If A then B, A holds, therefore B must hold

– **Resolution Inference:**

Applied to clausal form formulas $S = \bigwedge_i C_i$ where two clauses $C_i = L \vee \bigvee_j L_{i,j}, C_{i'} = \overline{L} \vee \bigvee_{j'} L_{i',j'}$ are resolved into one clause $C_{i|i'} = \bigvee_j L_{i,j} \vee \bigvee_{j'} L_{i',j'}$

– **Example**

$\dfrac{C \vee p \qquad \neg p \vee D}{C \vee D}$

$-C \vee D$ is the resolvent, p is the resolved atom

– **Factoring Inference:** $\dfrac{C \vee I \vee I}{C \vee I}$

Optimized DCFT

– **Naming Lemma** (on positive polarity)

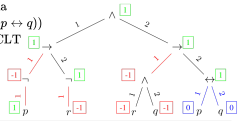
Given a set of formulas S and a formula B with only. pos polarity. Let n be variable not occuring in S nor in B.

Then $S \iff S' \cup \{n \rightarrow B\}$

– **Naming Lemma** (on negative polarity)

[...] B with only neg. polarity. [...] Then $S \iff S' \cup \{B \rightarrow n\}$

– **Example:** Given formula $(\neg p \rightarrow \neg r) \wedge ((r \wedge q) \rightarrow (p \leftrightarrow q))$ with polarity tree and DCLT



Name	Subformula	Definition	Clauses
			n_1
n_1	$(\neg p \rightarrow \neg r) \wedge ((r \wedge q) \rightarrow (p \leftrightarrow q))$	$n_1 \rightarrow (n_2 \wedge n_3)$	$\neg n_1 \vee n_2$ $\neg n_1 \vee n_3$
n_2	$\neg p \rightarrow \neg r$	$n_2 \rightarrow (\neg p \rightarrow \neg r)$	$\neg n_2 \vee p \vee \neg r$
n_3	$(r \wedge q) \rightarrow (p \leftrightarrow q)$	$n_3 \rightarrow (n_4 \rightarrow n_5)$	$\neg n_3 \vee \neg n_4 \vee n_5$
n_4	$r \wedge q$	$(r \wedge q) \rightarrow n_4$	$\neg r \vee \neg q \vee n_4$
n_5	$p \leftrightarrow q$	$n_5 \rightarrow (p \leftrightarrow q)$	$\neg n_5 \vee \neg p \vee q$ $\neg n_5 \vee p \vee \neg q$

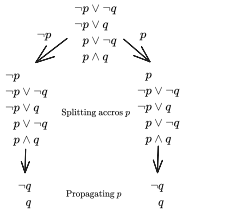
– **Splitting Algorithm** (with pure atoms)

```
procedure split(G)
parameters: function select
input: formula G
output: "satisfiable" or "unsatisfiable"
begin
  G := simplify_with_pure_atoms(G)
  if G = ⊤ then return "satisfiable"
  if G = ⊥ then return "unsatisfiable"
  (p, b) := select(G)
  case b of
  1 ⇒
    if split(G_p^⊤) ="satisfiable"
      then return "satisfiable"
      else return split(G_p^⊥)
  0 ⇒
    if split(G_p^⊥) ="satisfiable"
      then return "satisfiable"
      else return split(G_p^⊤)
end
```

– **DPLL** (splitting + propagation)

```
procedure DPLL(S)
input: set of clauses S
output: satisfiable or unsatisfiable
parameters: function select_literal
begin
  S := propagate(S)
  if S is empty then return satisfiable
  if S contains □ then return unsatisfiable
  L := select_literal(S)
  if DPLL(S ∪ {L}) = satisfiable
    then return satisfiable
    else return DPLL(S ∪ {L̄})
end
```

– **Example:**



Splitting and propagating q would result in formulas containing only the empty clause $\square$ which are always unsat

Tableau Inferences Rules

$$\frac{\mathbf{f} : A \vee B}{\mathbf{f} : A \mid \mathbf{f} : B} \qquad \frac{\mathbf{t} : A \vee B}{\mathbf{t} : A \mid \mathbf{t} : B} \qquad \frac{\mathbf{f} : A \wedge B}{\mathbf{f} : A \mid \mathbf{f} : B} \qquad \frac{\mathbf{t} : A \wedge B}{\mathbf{t} : A \\ \mathbf{t} : B}$$

$$\frac{\mathbf{f} : A \rightarrow B}{\mathbf{t} : A \\ \mathbf{f} : B} \qquad \frac{\mathbf{t} : A \rightarrow B}{\mathbf{f} : A \mid \mathbf{t} : B} \qquad \frac{\mathbf{f} : \neg A}{\mathbf{t} : A} \qquad \frac{\mathbf{t} : \neg A}{\mathbf{f} : A}$$

Given a formula G the first entry of the tableau is of the form $\mathbf{t} : G$. From there, all further entries are derivated using the inference rules.

– Rules like $\mathbf{t} : A \vee B$ produce two branches.

– A branch is closed if it contains a contradiction $\mathbf{f} : A$ and $\mathbf{t} : A$. The tableau is closed if all all branches are closed.

– If no more inference rules can be applied to a branch and it didn't close, then it provides an interpretation satisfying the formula

Derivations

Let $KB = \{C_1, \ldots, C_k\}$ be a set of *input (given) clauses*

A derivation (from KB) is a sequence of the form

$$\underbrace{C_1, \ldots, C_k}_{\text{Input clauses}}, \underbrace{C_{k+1}, \ldots, C_n, \ldots}_{\text{Derived clauses}}$$

such that for every $n \geq k + 1$

- C_n is a resolvent of C_i and C_j, for some $1 \leq i, j < n$, or
- C_n is a factor of C_i, for some $1 \leq i < n$.

The empty disjunction, or empty clause, is written as $\square$

A refutation (of KB) is a derivation from KB that contains $\square$

Refutation (using tableau)

To (dis)prove a statement $A \models B$, one can use the tableau rules on the formula $\neg(A \rightarrow B)$ until:

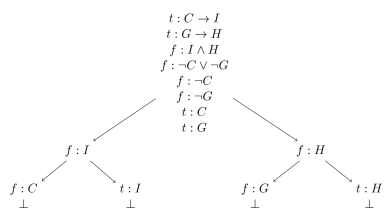
– The tableau is closed, in which case the statement holds.

– No more inferences can be applied, in which case the statement does not hold.

– **Example**

Given the statement $\underbrace{(C \rightarrow I) \wedge (C \rightarrow H) \wedge \neg(I \wedge H)}_{A} \models \underbrace{\neg C \vee G}_{B}$

One can (dis)prove this statement by using the tableu on $\neg(A \rightarrow B)$ i.e. $(C \rightarrow I) \wedge (G \rightarrow H) \wedge \neg(I \wedge H) \wedge \neg(\neg C \vee G)$



The tableau is closed and thus the entailment is proven

PL